

# Calibre Protocol — Complete Map

Architecture, cryptography, economics, and roadmap

Calibre Protocol Contributors

2026-09-23 · v0.8.9

## Contents

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| <b>1</b> | <b>Calibre Protocol — Complete Map</b>        | <b>2</b>  |
| 1.1      | Table of Contents . . . . .                   | 2         |
| <b>2</b> | <b>Part I — Foundations</b>                   | <b>2</b>  |
| 2.1      | 1. The problem . . . . .                      | 2         |
| 2.2      | 2. The approach . . . . .                     | 3         |
| <b>3</b> | <b>Part II — Architecture</b>                 | <b>3</b>  |
| 3.1      | 3. Q-UTXO state model . . . . .               | 3         |
| 3.2      | 4. QuantumLock . . . . .                      | 3         |
| 3.3      | 5. The four verification gates . . . . .      | 4         |
| 3.4      | 6. UTXO lifecycle . . . . .                   | 4         |
| <b>4</b> | <b>Part III — Cryptography</b>                | <b>4</b>  |
| 4.1      | 7. ML-DSA-44 . . . . .                        | 4         |
| 4.2      | 8. Lock binding . . . . .                     | 5         |
| 4.3      | 9. AEGIS context derivation . . . . .         | 5         |
| 4.4      | 10. Signature and verification flow . . . . . | 5         |
| <b>5</b> | <b>Part IV — Consensus</b>                    | <b>6</b>  |
| 5.1      | 11. Aura + GRANDPA . . . . .                  | 6         |
| 5.2      | 12. The validator set . . . . .               | 6         |
| 5.3      | 13. Finality . . . . .                        | 6         |
| <b>6</b> | <b>Part V — Economic layer</b>                | <b>7</b>  |
| 6.1      | 14. The fee market . . . . .                  | 7         |
| 6.2      | 15. Block rewards . . . . .                   | 7         |
| 6.3      | 16. Producer payout routing . . . . .         | 8         |
| 6.4      | 17. Staking . . . . .                         | 8         |
| <b>7</b> | <b>Part VI — History and status</b>           | <b>9</b>  |
| 7.1      | 18. The 58 commits . . . . .                  | 9         |
| 7.2      | 19. Phase map . . . . .                       | 9         |
| 7.3      | 20. Test inventory . . . . .                  | 10        |
| 7.4      | 21. Decision log . . . . .                    | 11        |
| <b>8</b> | <b>Part VII — Forward</b>                     | <b>11</b> |
| 8.1      | 22. Open debt . . . . .                       | 11        |
| 8.2      | 23. Unscheduled work . . . . .                | 12        |

|     |                                       |    |
|-----|---------------------------------------|----|
| 8.3 | 24. Revised plan . . . . .            | 12 |
| 8.4 | Appendix A — File layout . . . . .    | 13 |
| 8.5 | Appendix B — Build and test . . . . . | 14 |
| 8.6 | Appendix C — Glossary . . . . .       | 14 |

# 1 Calibre Protocol — Complete Map

A post-quantum Layer-1 blockchain with real FIPS 204 ML-DSA-44 verification on-chain.  
Version 0.8.9 · 68 tests passing · commit 674cb53

---

## 1.1 Table of Contents

Part I — Foundations 1. The problem 2. The approach

Part II — Architecture 3. Q-UTXO state model 4. QuantumLock 5. The four verification gates 6. UTXO lifecycle

Part III — Cryptography 7. ML-DSA-44 8. Lock binding 9. AEGIS context derivation 10. Signature and verification flow

Part IV — Consensus 11. Aura + GRANDPA 12. The validator set 13. Finality

Part V — Economic layer 14. The fee market 15. Block rewards 16. Producer payout routing 17. Staking

Part VI — History and status 18. The 58 commits 19. Phase map 20. Test inventory 21. Decision log

Part VII — Forward 22. Open debt 23. Unscheduled work 24. Revised plan

---

## 2 Part I — Foundations

### 2.1 1. The problem

Every major blockchain today — Bitcoin, Ethereum, Solana, Polkadot — relies on elliptic-curve cryptography (ECDSA or Ed25519). A sufficiently large quantum computer running Shor’s algorithm can invert these in polynomial time. Estimates for a cryptographically relevant quantum computer range from 8 to 30 years. Whenever it arrives, every wallet with an exposed public key becomes drainable.

Beyond quantum, blockchains inherit three systemic problems:

| Problem               | Cost                                  |
|-----------------------|---------------------------------------|
| Seed-phrase fragility | USD 1B+/year in losses                |
| Bridge honeypots      | Ronin 625M, Wormhole 320M, Nomad 190M |
| Blind signing         | Users sign opaque bytecode            |

## 2.2 2. The approach

Calibre is PQ-first. It does not migrate to post-quantum cryptography — it starts there. Four design choices:

1. Quantum-UTXO state model — value is an unspent output bound to a PQ lock
  2. Real on-chain ML-DSA-44 — verification inside the WASM runtime
  3. Four independent verification gates — mempool, state, runtime, crypto
  4. Fast-Path conflict detection — UTXO conflicts are syntactic, so double-spends reject in microseconds
- 

## 3 Part II — Architecture

### 3.1 3. Q-UTXO state model

In the account model (Ethereum), a balance is a number stored next to an address. In the UTXO model (Bitcoin), a balance is the sum of unspent outputs a key can unlock.

Calibre's state is a single map:

```
UtxoSet: Map<H256, Utxo<Balance>>
```

```
struct Utxo<Balance> {  
    value: Balance,  
    lock: QuantumLock,  
}
```

There is no AccountId in state. The state is purely a map from UTXO identifier to output.

UTXO identifier:

```
utxo_id = blake2_256(origin_tx_hash || output_index)
```

where || is byte concatenation.

Why UTXO, not accounts. In account chains, two transactions conflict only if execution discovers the conflict. In UTXO chains, two transactions conflict if and only if they share an input — a syntactic property, checkable in microseconds without executing anything.

This enables:

- Mempool-level double-spend rejection
- Parallel verification of non-conflicting transactions
- Deterministic conflict detection independent of node state

### 3.2 4. QuantumLock

```
enum QuantumLock {  
    AegisThreshold([u8; 32]), // blake2_256(ML-DSA-44 pk)  
    SingleSig([u8; 32]),      // test-only, rejected by verifier  
}
```

The whitepaper names a third variant — HybridPqClassical { pq\_lock, classical\_lock } — for forkless migration. It is not implemented. Only two variants exist today.

AegisThreshold is the production lock. SingleSig exists for legacy test fixtures and is rejected by verify\_aegis\_transaction with LockTypeMismatch.

### 3.3 5. The four verification gates

Every signed UTXO spend passes through four independent gates:

| # | Gate    | Location                 | Rejects                                    |
|---|---------|--------------------------|--------------------------------------------|
| 1 | Mempool | validate_unsigned        | Double-spends, via per-input conflict tags |
| 2 | State   | execute_utxo_tx          | Replays, via UtxoSet::contains_key         |
| 3 | Runtime | verify_aegis_transaction | Lock mismatch                              |
| 4 | Crypto  | MLDsaKeyPair::verify     | Forgeries                                  |

Each gate catches a distinct class of attack. Failing any gate rejects the transaction. An attacker must defeat all four.

### 3.4 6. UTXO lifecycle

Created --> Unspent --> Spent (removed from set)

A UTXO is created by:

- sudo\_mint(value, lock) — genesis / testnet faucet
- mint\_utxo(value, lock) — block rewards, producer payouts, unbonds

A UTXO is destroyed by:

- execute\_utxo\_tx — a signed spend (inputs consumed, outputs created)
- UtxoConsumer::consume\_with\_witness — stake bond

Total issuance invariant (currently violated, see Section 22):

TotalIssuance = sum of value over all UtxoSet entries

The invariant is supposed to hold. On the execute\_utxo\_tx path it does not, because that path never decrements TotalIssuance on input removal.

---

## 4 Part III — Cryptography

### 4.1 7. ML-DSA-44

Standard: NIST FIPS 204 (finalized August 2024) Family: CRYSTALS-Dilithium, lattice-based Library: dilithium-rs 0.4 (pure Rust, no\_std compatible) Security level: ML-DSA-44 (NIST Level 2, ~128-bit classical)

Key sizes:

| Element    | Bytes |
|------------|-------|
| Public key | 1312  |
| Secret key | 2560  |
| Signature  | 2420  |

Verification cost on target hardware: ~208 microseconds per signature.

## 4.2 8. Lock binding

The critical binding between a UTXO and its owner:

```
lock = blake2_256(pub_key)
```

The verifier enforces:

```
blake2_256(witness.pub_keys[0]) == utxo.lock
```

This is gate 3. Without it, anyone could present their own valid signature over someone else's UTXO. With it, the signature is bound to the exact key that owns the output.

## 4.3 9. AEGIS context derivation

The whitepaper describes a context derivation that mixes device attestation:

```
context = Shake256(  
    biometric_entropy ||  
    attestation.hardware_signature ||  
    attestation.app_binary_hash ||  
    attestation.backend_challenge ||  
    time_drift  
)
```

Reality: `verify_aegis_transaction` calls `verify_dilithium_signature(pk, msg, sig)` with context `b""` — the empty byte string. The attestation fields are decoded but never verified. The AEGIS derivation exists in the code as `derive_context` but is not on the verification path.

Consequence: the “hardware-backed” claim is currently decorative. Either verify attestation (AppAttest / Play Integrity server-side) or remove the claim from the whitepaper.

## 4.4 10. Signature and verification flow

Sign (client side, pq-signer):

```
kp      = MLDsaKeyPair::generate(ML_DSA_44)  
pk      = kp.public_key()  
lock    = blake2_256(pk)  
msg     = (inputs, outputs).encode()  
sig     = kp.sign(msg, ctx = b"")  
witness = AegisWitness {  
    pub_keys: [pk],  
    aggregated_signature: sig.as_bytes(),  
    attestation: { ... },  
    time_drift: 0,  
}
```

Verify (runtime side, gate 4):

```
payload = (inputs, outputs).encode()  
MLDsaKeyPair::verify(  
    pub_key = witness.pub_keys[0],  
    sig     = DilithiumSignature::from_bytes(witness.aggregated_signature),  
    message = payload,  
    ctx     = b"",  
    mode    = ML_DSA_44,  
) -> bool
```

Transaction encoding:

```
Transaction {
    inputs: Vec<TransactionInput>, // (tx_hash, output_index)
    outputs: Vec<TransactionOutput>, // (value, lock)
    pq_signature: Vec<u8>, // unused; witness carries the sig
    witness: Vec<u8>, // SCALE-encoded AegisWitness
}
```

---

## 5 Part IV — Consensus

### 5.1 11. Aura + GRANDPA

Aura (Authority Round) produces blocks. Slot-based, deterministic rotation.

GRANDPA (GHOST-based Recursive ANcestor Deriving Prefix Agreement) finalizes. Byzantine-fault-tolerant with  $2/3+1$  threshold.

Slot duration: 6 seconds. Finality lag:  $\sim 2$  blocks (12 seconds).

### 5.2 12. The validator set

Testnet authority set:  $n = 7, f = 2$ .

BFT threshold for finality:

$\text{threshold} = \text{ceil}(2n/3) = \text{ceil}(14/3) = 5 \text{ of } 7$

Aura author selection:

$\text{author\_index} = \text{slot} \% \text{authorities.len}()$

The author of slot  $s$  is  $\text{Authorities}[s \bmod n]$ .

### 5.3 13. Finality

Verified on 7-validator Docker testnet (commit faed330):

| Property                 | Result                 |
|--------------------------|------------------------|
| Full peer mesh           | 6/6 peers connected    |
| GRANDPA finality engaged | at 5/7 threshold       |
| Latency tolerance        | 1s RTT survived        |
| Throughput parity        | 169 TPS single = multi |

Caveat: single-node throughput shows empty blocks while txs pend (Section 23.4). The 169 TPS number is measured, but the mechanism behind the limiter is unknown.

---

## 6 Part V — Economic layer

### 6.1 14. The fee market

Every transaction pays an implicit fee:

```
fee = sum(inputs) - sum(outputs)
```

The minimum acceptable fee scales with transaction size:

```
minimum_fee(inputs, outputs) =  
    base_fee + per_in_out_fee * (inputs + outputs)
```

Runtime constants:

```
base_fee          = 1,000      (per transaction)  
per_in_out_fee    = 100        (per input + per output)  
min_base_fee      = 500        (floor)  
max_base_fee      = 1,000,000  (ceiling)
```

So a 1-in/1-out transaction has  $\text{minimum\_fee} = 1000 + 100 \cdot 2 = 1200$ .

Dynamic base fee (EIP-1559-style). Adjusted in `on_finalize`, targeting 50% block fullness:

```
if actual > target:  
    ratio = min((actual - target) / target, 1.0)  
    change = max_change_pct * ratio  
    new     = current * (1 + change)
```

```
if actual < target:  
    ratio = min((target - actual) / target, 1.0)  
    change = max_change_pct * ratio  
    new     = current * (1 - change)
```

```
new = clamp(new, min_base_fee, max_base_fee)
```

With `max_change_pct = 12%` (0.12).

Fee split. The base fee portion is split:

```
producer_cut = fee * 0.50  
treasury_cut = fee * 0.30  
burn_cut     = fee * 0.20
```

Burn absorbs any rounding remainder.

Priority fee. Anything paid above `minimum_fee` is a priority fee:

```
priority_fee = max(paid - minimum_fee, 0)
```

Priority fees route 100% to the producer. No split.

### 6.2 15. Block rewards

Per block: 5.3 CAL. Halving every 4 years. Supply cap 1,000,000,000 CAL. Genesis allocation 777,000,000 CAL.

Split:

```
producer_share = reward * 0.70  
treasury_share = reward * 0.30
```

Emissions schedule:

year 1: 5.3 CAL/block x 5,256,000 blocks/year = 27.86M CAL  
year 5: 2.65 CAL/block = 13.93M CAL  
year 9: 1.325 CAL/block = 6.96M CAL

...

asymptote: 504,580,000 CAL (50.46% of cap)

Documentation drift: docs/ECONOMIC\_MODEL.md and the whitepaper were written with 12 CAL/block. The final tokenomics locked 5.3. Both docs need updating.

## 6.3 16. Producer payout routing

Every block, the producer's accumulated earnings settle in `on_finalize`:

```
earnings = ProducerAccumulated      // 50% of fees + 70% of rewards
          + PriorityFeeAccumulated   // 100% of tips
          + ProducerPending[author]  // carried from prior blocks
```

```
if ProducerLocks[author] exists:
    mint UTX0(earnings, ProducerLocks[author])
    clear accumulators and pending
else:
    ProducerPending[author] = earnings // never burn
    clear accumulators
```

Invariant: a producer's earnings are never destroyed. If they haven't registered a lock, the amount waits — indefinitely, if necessary — until they call `register_producer_lock`.

This follows the Ethereum 0x00 withdrawal-credential precedent and the Cosmos outstanding-commission model. Both accrue; neither burns.

## 6.4 17. Staking

Model: burn-and-record.

`stake[account]: Map<AccountId, Balance>`

Bond:

```
consume UTX0s with witness -> amount
stake[account] += amount
TotalStaked    += amount
```

Unbond:

```
assert stake[account] >= amount
stake[account] -= amount
TotalStaked    -= amount
mint UTX0(amount, caller_supplied_lock)
```

No unbonding period. No slashing. No gating.

Staking today is a ledger, not a security mechanism. To gate consensus (validator selection, reward sharing), several things are missing — see Section 23.1.

## 7 Part VI — History and status

### 7.1 18. The 58 commits

Chronological, grouped by phase. Full detail in docs/PROJECT\_LOG.md.

2026-09-21 — Core engine, testnet, ZK design

dc752b0 Calibre Protocol v0.6.0-testnet — Q-UTX0 + ML-DSA-44 + Docker + docs  
3fcb533 phase 6.6: working 7-validator testnet (n=7 f=2 BFT)  
2d1734d phase 7.0: light node + ZK state proof design document  
9dcad9b phase 7.0: RISC Zero selected (SP1 abandoned)  
9b64644 phase 7.1: Merkle root tracking in pallet-qutxo  
6330be9 phase 7.2: extract Merkle logic to calibre-merkle  
dacfbcd phase 7.2: end-to-end UTX0 inclusion proof verified  
fb923a7 phase 7.2d: QutxoApi crate + workspace wiring  
a21e0b7 phase 7.2d: qutxo\_getInclusionProof RPC endpoint  
fa63111 phase 7.2d: host fetches inclusion proof from live RPC

2026-09-22 — Light client, ZK verifier, anti-spam, finality

21ac57a phase 7.5: calibre-light v0.1 — trust tier 1 light client  
5b68d4c phase 7.5 v0.4: freshness layer + tier-1/tier-2 upgrade  
112b199 phase 7.6-prep: Android cross-compile verified  
d7d61fe 7.4: use risc0-zkvm 3.0.6 for on-chain verification (unblocked)  
4e6c0ab anti-spam: full ML-DSA verify in validate\_unsigned + 9 tests  
f8e9e23 weight: honest weights for execute\_utxo\_tx  
f2e6c4b validator: 7-authority staging chain spec + session-key wiring  
faed330 deploy: 7-validator docker testnet

2026-09-23 — Fee market, staking, tests, docs

06766f9 fee-market: scaffold pallet-calibre-fees (Phase 8.1)  
530cfce fee-market: wire FeeHandler into qutxo (Phase 8.2)  
875c75a fee-market: reject under-priced txs at pool admission (Phase 8.3)  
8b5e264 fee-market: dynamic base fee (Phase 8.4)  
301a45d fee-market: block rewards + tokenomics + economics  
e2912eb tokenomics: final 777M genesis (5.3 CAL/block)  
16da3d6 fee-market: producer payout routing (Phase 8.6)  
a73f95f stake: bond/unbond pallet (Phase 8.7)  
b47530a test: real ML-DSA-44 witness harness + fee-rejection E2E (Phase 8.8)  
ae8a5a1 bench: bench txs now pay fees (Phase 8.9)  
674cb53 docs: add PROJECT\_LOG.md — commit-by-commit record

### 7.2 19. Phase map

| Group       | Phase   | Name                                                | Status |
|-------------|---------|-----------------------------------------------------|--------|
| Engine      | 1-5     | Q-UTX0, ML-DSA-44,<br>Fast-Path, Console,<br>Docker | done   |
| Publication | 6.1-6.5 | Testnet, faucet,<br>whitepaper,<br>diagrams, GitHub | done   |
| Consensus   | 6.6     | Multi-validator<br>testnet (n=7 f=2)                | done   |

| Group      | Phase | Name                                        | Status                  |
|------------|-------|---------------------------------------------|-------------------------|
| Economic   | 6.7   | Fee market + staking                        | done (bond/unbond only) |
| ZK / light | 7.0   | Design doc + RISC Zero selection            | done                    |
| ZK / light | 7.1   | Merkle root tracking                        | done                    |
| ZK / light | 7.2   | calibre-merkle + inclusion proof            | done                    |
| ZK / light | 7.2d  | QutxoApi crate + RPC endpoint               | done                    |
| ZK / light | 7.4   | On-chain RISC Zero verifier                 | done                    |
| ZK / light | 7.5   | calibre-light v0.1-v0.4                     | done                    |
| ZK / light | 7.6   | Android app                                 | in progress             |
| Economic   | 8.1   | pallet-calibre-fees scaffold                | done                    |
| Economic   | 8.2   | FeeHandler wired into qutxo                 | done                    |
| Economic   | 8.3   | Pool rejects under-priced txs               | done                    |
| Economic   | 8.4   | Dynamic base fee                            | done                    |
| Economic   | 8.5   | Block rewards                               | done                    |
| Economic   | 8.6   | Producer payout routing                     | done                    |
| Economic   | 8.7   | Stake pallet                                | done                    |
| Testing    | 8.8   | Real ML-DSA witness harness                 | done                    |
| Testing    | 8.9   | Bench pays fees                             | done                    |
| Audit      | 9     | Benchmark weights, fuzz, try-runtime, audit | partial                 |
| Bridge     | 10    | ZK Intent Bridge                            | not started             |
| Privacy    | 11    | ZkShield                                    | not started             |
| Launch     | 12    | Mainnet                                     | not started             |

### 7.3 20. Test inventory

Total: 68 tests, 0 failed, 0 ignored.

| Crate               | Count | Coverage                                                                                                    |
|---------------------|-------|-------------------------------------------------------------------------------------------------------------|
| pallet-qutxo        | 21    | Conservation, double-spend, Merkle root, 11 validate_unsigned rejections, 3 fee E2E, real ML-DSA-44 signing |
| pallet-calibre-fees | 24    | 50/30/20 split, base fee dynamics, block rewards, priority fee, 6 producer-settle cases                     |

| Crate           | Count | Coverage                                                                                           |
|-----------------|-------|----------------------------------------------------------------------------------------------------|
| pallet-stake    | 10    | Bond, consumer failure, input bound, accumulate, unbond+mint, over-balance, full unbond, isolation |
| calibre-merkle  | 7     | Path construction, root computation, insertion-order determinism                                   |
| pallet-template | 4     | Template baseline                                                                                  |
| runtime         | 2     | Integrity test, genesis build                                                                      |

Run: `SKIP_WASM_BUILD=1 cargo test --workspace`

## 7.4 21. Decision log

| Decision               | Choice                              | Rationale                                     | Commit  |
|------------------------|-------------------------------------|-----------------------------------------------|---------|
| Fee split              | 50/30/20                            | Balances incentive, runway, deflation         | 06766f9 |
| Reward split           | 70/30                               | Validators bear cost; treasury captures slice | 301a45d |
| Base fee               | EIP-1559, 12%, 500-1M               | Predictable fees                              | 8b5e264 |
| Genesis                | 777M, 5.3/block, 4y halving, 1B cap | Self-funded viable on EUR 13K                 | e2912eb |
| Payout timing          | Once per block                      | Matches existing cadence                      | 16da3d6 |
| Unregistered producer  | Accrue pending, never burn          | Ethereum 0x00, Cosmos precedent               | 16da3d6 |
| Author resolution      | FindAuthor config type              | No new pallet, mockable                       | 16da3d6 |
| Staking model          | Burn-and-record                     | Simplest; upgradeable to time-locked          | a73f95f |
| Staking / payout       | Separate concerns                   | Orthogonal                                    | a73f95f |
| Cardano liquid staking | Future, not built                   | Needs dual-key redesign                       | a73f95f |

## 8 Part VII — Forward

### 8.1 22. Open debt

Three items. Each small, independent, none blocks functionality.

#### 22.1 TotalIssuance stale on `execute_utxo_tx`

The `UtxoConsumer` trait (used by stake) decrements correctly. The original path doesn't. Fix candidate: decrement symmetric to `consume_with_witness`, or recompute from `UtxoSet::iter()` in `on_finalize`.

## 22.2 AuraFindAuthor slot-to-author math untested

Written, compiles, runs. Can't be tested with the u64 mock AccountId. Needs a runtime integration test.

## 22.3 Treasury accounting-only

TreasuryAccumulated is written but never minted. 30% of fees and 30% of block rewards accumulate into a storage value that is functionally burned.

## 8.2 23. Unscheduled work

Discovered by testing. None has a phase.

### 23.1 Stake gating

Bond exists. It gates nothing. Missing:

- Validator selection by stake weight (consensus change)
- Unbonding period (flash-stake protection)
- Slashing conditions (risk, without which stake secures nothing)
- Reward distribution to delegators

### 23.2 Dual-key staking

Cardano-style liquid staking needs payment key + staking key as separate concepts. Changes key derivation, delegation, and consensus stake-weighting. A protocol redesign, not a feature.

### 23.3 Hardware attestation

Fields exist, never verified. Either verify (AppAttest / Play Integrity) or drop the claim.

### 23.4 Throughput limiter

From docs/BENCHMARK.md: "empty blocks appear while txs pend." Blocks use 12.8% refTime / 27.7% bytes, yet pool has pending transactions. Binding limit is upstream of block weight. Cause unknown.

### 23.5 Whitepaper refresh

Version 0.6.0 predates the economic layer. Overstates A.E.G.I.S., forkless migration, seedphrase elimination.

## 8.3 24. Revised plan

Keep the 11 public phases. Add sub-phases for the unscheduled work.

| Phase   | Name                    | Status | Notes                                                                                                                                   |
|---------|-------------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------|
| 1-5     | Core engine             | done   |                                                                                                                                         |
| 6.1-6.5 | Testnet and publication | done   |                                                                                                                                         |
| 6.6     | Multi-validator testnet | done   |                                                                                                                                         |
| 6.7     | Fee market + staking    | done   | bond/unbond only<br>validator selection,<br>unbonding, slashing<br>withdraw extrinsic +<br>governance<br>delegator rewards,<br>APY live |
| 6.8     | Stake gating            | new    |                                                                                                                                         |
| 6.9     | Treasury governance     | new    |                                                                                                                                         |
| 6.10    | Economic model v1       | new    |                                                                                                                                         |

| Phase | Name                       | Status      | Notes                                                                                                                          |
|-------|----------------------------|-------------|--------------------------------------------------------------------------------------------------------------------------------|
| 7     | Light nodes + ZK proofs    | in progress | library + verifier shipped; mobile app pending<br>weights done; fuzz, try-runtime, audit<br>resolve empty-blocks-while-pending |
| 8     | Audit preparation          | in progress |                                                                                                                                |
| 8.5   | Throughput investigation   | new         |                                                                                                                                |
| 9     | ZK Intent Bridge           | not started |                                                                                                                                |
| 10    | ZkShield                   | not started | Cardano model — protocol redesign                                                                                              |
| 10.5  | Dual-key staking           | new         |                                                                                                                                |
| 11    | Final tokenomics + mainnet | not started |                                                                                                                                |
| 11.5  | Whitepaper v1.0            | new         | full reconciliation                                                                                                            |

Key structural change: Phase 6.6 “done” was premature. Consensus works; economic consensus does not. Phase 6 stays open until 6.10 ships.

## 8.4 Appendix A — File layout

```
calibre-template/
  crates/
    aegis-crypto/      ML-DSA-44 wrapper, verify_aegis_transaction
    calibre-light/     light client (v0.4, UniFFI Swift/Kotlin)
    calibre-merkle/    Merkle path + root
    qutxo-rpc-api/     RPC types for qutxo_getInclusionProof
    risc0-host/        RISC Zero verifier host
  docs/
    whitepaper/        4-part spec (v0.6.0)
    diagrams/          7 mermaid diagrams (01-07)
    SESSION_STATE.md   current phase + debt
    PROJECT_LOG.md     commit-by-commit record
    PROTOCOL_MAP.md    whitepaper-vs-reality reconciliation
    CALIBRE_MAP.md     this document
  pallets/
    qutxo/             core UTXO state machine
    calibre-fees/      fee market, rewards, payouts
    stake/             bond/unbond
    zk-verifier/       on-chain RISC Zero
    template/          baseline
  runtime/            Substrate runtime wiring
  node/              Substrate node
  tools/
    bench/            TPS benchmark
    tokenomics/       Python emission model
    pq-signer/        ML-DSA-44 CLI signer
  deploy/            Docker + WAN scripts
```

## 8.5 Appendix B — Build and test

```
# Native build (fast)
SKIP_WASM_BUILD=1 cargo build

# Full runtime (WASM)
cargo build -p solochain-template-runtime

# All tests
SKIP_WASM_BUILD=1 cargo test --workspace

# Docker testnet
docker compose up
```

## 8.6 Appendix C — Glossary

| Term             | Meaning                                             |
|------------------|-----------------------------------------------------|
| AEGIS            | The ML-DSA-44 wrapper layer in calibre-aegis-crypto |
| Aura             | Slot-based block production consensus               |
| CRQC             | Cryptographically Relevant Quantum Computer         |
| GRANDPA          | Finality gadget, 2/3+1 BFT threshold                |
| ML-DSA-44        | NIST FIPS 204 lattice signature, Level 2            |
| Q-UTXO           | Quantum-UTXO — Calibre’s extended UTXO model        |
| QuantumLock      | The lock enum on each UTXO                          |
| Shor’s algorithm | Quantum algorithm that breaks ECDSA/Ed25519         |
| UTXO             | Unspent Transaction Output                          |

---

Calibre Protocol is a research prototype. Not audited. Not for production use. This document is v0.8.9 and reflects commit 674cb53.